Enabling Large-VRAM GPUs on the Dell Precision 5820

Unlocking the hidden MMIO High Granularity setting so big-BAR cards will POST

Field	Value
Hardware	Dell Precision 5820 Tower (Intel C422 / Xeon W board)
BIOS	V0 AMI, version 2.48.0
Validated GPU	Intel Arc Pro B70 (32 GB VRAM)
Method	Direct UEFI NVRAM variable edit (no BIOS re-flash)
Document version / date	1.0 — written from a verified, working change

1. What this guide does

Dell removed the high-memory-mapped-I/O (MMIO) window controls from the Precision 5820's visible BIOS menu. The settings still exist in the firmware — they are simply hidden. Because of that, a GPU with a large memory aperture (a *Base Address Register*, or BAR) cannot be allocated address space at power-on. The machine appears to fail: with the card installed it boots straight into BIOS Setup and will not load the operating system. Enabling “Above 4G Decoding” alone does **not** fix this, because that switch only grants permission to map above the 4 GB line; it does not size the window.

This guide shows how to reach the hidden **MMIO High Granularity Size** setting through the UEFI Shell and widen the window so large cards POST and boot normally. It is written for the **Intel Arc Pro B70 (32 GB)**, but the same procedure applies to any large-BAR card (professional GPUs, compute accelerators, etc.).

NOTE This procedure edits a single firmware variable. It does **not** re-flash or patch the BIOS, so Dell's signed-firmware protection is never challenged. Every change is fully reversible.

2. The root cause, in one line

The variable `SocketCommonRcConfig` ships with MMIO High Granularity set to **1G**. That caps the entire high-MMIO window at 32 × 1G = 32 GB and hands each card only 1 GB by default — far too little for a 32 GB card's BAR. Raising the granularity gives each card enough room, and the machine POSTs.

3. Read before you start (safety)

WARNING You are editing firmware NVRAM. A wrong value can stop the machine from POSTing. Follow the backup step, change only the one field described here, and keep the restore command from Section 9 on hand before you write anything.

- **Back up first.** Record the variable's current contents (Section 6) before any change.
- **Change one field only.** Use the exact commands provided. Do not improvise other bytes.
- **You have two ways back.** The restore command in Section 9, or a CMOS/RTCRST clear, both return the variable to its factory default.
- **Match your BIOS version.** Everything here was verified on BIOS 2.48.0. After any future BIOS update, re-verify before re-applying (Section 10).

4. What you need

- A USB stick (4 GB or larger), formatted FAT32.
- The signed UEFI Shell binary `shellx64.efi` from the UEFI-Shell releases page (github.com/pbatard/UEFI-Shell).
- Physical access to the workstation and its keyboard/monitor.
- The GPU removed for now, so the machine boots normally to the Shell.

The key facts this guide is built on (verified on this machine):

Item	Value
Variable name	`SocketCommonRcConfig`
GUID	`4402ca38-808f-4279-bcec-5baf8d59092f`
Variable size	0x73 (115 bytes)
Field to change	MMIO High Granularity Size
Field location	byte offset 0x04, 2 bytes, little-endian
Factory default	0x0000 (= 1G)

5. Build the UEFI Shell USB and boot it

1. Format the USB stick as FAT32.
2. On the stick, create the folder path `EFI\BOOT` and place `shellx64.efi` inside it, renamed to `BOOTX64.EFI`. Final path: `USB:\EFI\BOOT\BOOTX64.EFI`.
3. With the GPU removed, plug the USB into the 5820 and power on.
4. Tap **F12** repeatedly to open the one-time boot menu, and select the USB entry (listed as "UEFI: …"). The Shell loads to a `Shell>` prompt.
5. Identify the USB filesystem: type `fs0:` then `ls`. If you see your EFI folder, you're on the right drive; if not, try `fs1:` or `fs2:`.

> **NOTE** Optional, to make the hex easier to read on screen: `mode 80 25` switches to the largest-text display mode.

6. Back up the current variable (required)

Read the variable and save a copy before changing anything. Run:

```
dmpstore -guid 4402ca38-808f-4279-bcec-5baf8d59092f SocketCommonRcConfig
```

You should see `DataSize = 0x73` and a hex dump whose first line reads:

```
00000000: 00 00 00 00 00 00 01 02-00 01 00 00 0E 06 00 00
```

To also save a file copy onto the USB stick (recommended), run:

```
dmpstore -guid 4402ca38-808f-4279-bcec-5baf8d59092f SocketCommonRcConfig -s fs0:\socket_backup.bin
```

NOTE If your first line differs from the one shown above, your BIOS version or configuration is different from the one this guide was verified on. Stop and re-derive the offset (Appendix A) before proceeding.

7. Choose the granularity value

Pick the smallest option whose **per-card window** is greater than your card's VRAM/BAR. For the 32 GB Arc Pro B70, that is **64G**. The same 64G setting comfortably covers one, two, or more B70s, because the total window it opens is 2 TB.

Firmware option	Value @ 0x04	Per-card window	Total window (32×)	Use for
16G	02 00	16 GB	512 GB	cards with BAR up to 16 GB only
64G	**03 00**	**64 GB**	**2 TB**	**32 GB cards — recommended for Arc Pro B70**
256G	04 00	256 GB	8 TB	very large BARs (see caution below)

NOTE There is **no 128G option** in this firmware — the rungs are 1G, 4G, 16G, 64G, 256G, 1024G. The default 1G and the 4G option are too small for a 32 GB card. **Avoid 1024G**: its window would run past this CPU's 64 TB physical-address ceiling. **256G is borderline**: its 8 TB window plus the default 56T base lands right at the 64 TB ceiling, so only use 256G if you also lower MMIO High Base (offset 0x00) — not needed for the B70.

NOTE On a Precision 7920 (a different, server-class platform), a 512G option exists and works; the 5820 does not offer it, and it is unnecessary for these cards.

8. Write the setting

For the Arc Pro B70 (recommended **64G**), run the command below. It rewrites the full variable with only offset 0x04 changed from 00 to 03:

```
setvar SocketCommonRcConfig -guid 4402ca38-808f-4279-bcec-5baf8d59092f -bs -rt -nv
=H000000003000102000100000E06000000000
```

```
setvar SocketCommonRcConfig -guid 4402ca38-808f-4279-bcec-5baf8d59092f -bs -rt -nv
=H00000000020001020001000000E06000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000
```

256G option:

```
setvar SocketCommonRcConfig -guid 4402ca38-808f-4279-bcec-5baf8d59092f -bs -rt -nv
=H00000000040001020001000000E06000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000
```

It is critical that you enter these commands EXACTLY as presented. Count the zeros. Include spaces where there are spaces and do not insert spaces where there are none. If you fail to follow this instruction, the command will NOT work.

9. Verify, then recover if ever needed

Verify the change

Re-read the variable:

```
dmpstore -guid 4402ca38-808f-4279-bcec-5baf8d59092f SocketCommonRcConfig
```

For the 64G setting, the first dump line should now read `00 00 00 00 03 00 01 02 ...` — that is, the fifth byte is now `03`.

Restore to factory default

If anything misbehaves, remove the card, boot back to the Shell, and run this to set the variable back to its 1G default:

```
setvar SocketCommonRcConfig -guid 4402ca38-808f-4279-bcec-5baf8d59092f -bs -rt -nv
=H00000000000001020001000000E06000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000
```

> **NOTE** A CMOS/RTCRST clear also restores the default, but it wipes all of your other BIOS settings. The command above is cleaner because it leaves everything else untouched.

10. Install the GPU and test

6. Power the machine fully off after the write and verify steps.
7. Install the GPU in a full **x16, slot** and connect power (slot 2 or slot 4 on the 5820). Start with a single card to keep the first test clean; add the second once the first is confirmed.
8. Power on. The machine should now complete POST and boot into the operating system instead of dropping into BIOS Setup. As is typical with the 5820, my machine powered itself off and on a few times before I saw the DELL splash screen. That is normal Dell behavior. Let it run through these cycles until you see the splash screen and subsequent boot loader processes.

RESULT If it boots to the OS with the card present, the fix is working. Confirm the card is seen by the OS (for example, `lspci | grep -i vga` on Linux).

WARNING If it still drops to Setup, return to the Shell (card out) and re-run the `dmpstore` read to confirm the 03 held. If it held but the card still won't POST, step the granularity up one rung (256G, with the MMIO High Base caution in Section 7) and retest.

11. After a future BIOS update

A BIOS update can change the layout of `SocketCommonRcConfig` — the size or a field offset can move. Updating from 2.47-era to **2.48.0** did not change anything here, but do not assume that for future versions. After any BIOS update:

9. Re-run the Section 6 read and confirm the size is still 0x73 and the first line still matches.
10. If the size or first line changed, re-derive the offset with the method in Appendix A before writing.
11. Only then re-apply the granularity setting (the variable typically returns to its 1G default after an update).

Appendix A — How the offset was found

This lets other owners verify the offset on a different BIOS version. The work was done on a separate Linux PC against Dell's BIOS update file (`Precision_5820_2.48.0.exe`).

12. **Extract the Dell package.** Use platomav's BIOSUtilities (the Dell PFS extractor) to unpack the .exe into its firmware components. The C422 board's image is the “System BIOS with BIOS Guard [V0 AMI]” bin; the X299 board's is “System BIOS [V0 AMI].”
13. **Decompress the volumes.** Run the BIOS bin through uefi-firmware-parser to recursively extract and decompress its firmware volumes, exposing the setup forms.
14. **Decode the IFR.** Run IFRExtractor-RS on the Setup firmware volume to produce human-readable Internal Forms Representation (IFR) text.
15. **Find the field.** Search the IFR for “MMIO High Granularity Size.” It resolves to a question in VarStore 0x2, which is defined as the SocketCommonRcConfig EFI variable (GUID 4402ca38-…, size 0x73). The question sits at VarOffset 0x04, with options 1G/4G/16G/64G/256G/1024G mapping to values 0–5.
16. **Cross-check against the live variable.** Every non-zero byte in the machine's live dump matched a defaulted IFR question at the same offset (MMIO High Base at 0x00, granularity at 0x04, Isoc at 0x08, NUMA at 0x09, and so on), confirming the layout before any write.

Appendix B — Quick command reference

All commands run at the UEFI Shell prompt. Substitute your USB filesystem (fs0:, fs1:, …) where shown.

Purpose	Command
Read variable	`dmpstore -guid 4402ca38-808f-4279-bcec-5baf8d59092f SocketCommonRcConfig`
Save backup to USB	`dmpstore -guid 4402ca38-808f-4279-bcec-5baf8d59092f SocketCommonRcConfig -s`

Purpose	Command
	`fs0:\socket_backup.bin`
Set 64G (B70)	`setvar … -bs -rt -nv =H0000000003000102000100000E0600… (offset 0x04 = 03)`
Set 16G	`setvar … -bs -rt -nv =H0000000002000102000100000E0600… (offset 0x04 = 02)`
Set 256G	`setvar … -bs -rt -nv =H0000000004000102000100000E0600… (offset 0x04 = 04)`
Restore default (1G)	`setvar … -bs -rt -nv =H0000000000000102000100000E0600… (offset 0x04 = 00)`

NOTE The "…" in the table is shorthand only; use the full commands from Sections 8 and 9 when actually typing them.

Prepared as a working, verified reference. Use at your own risk; firmware edits carry inherent risk. Shared in the spirit of helping fellow Precision owners run modern large-VRAM GPUs.